# Mini-guía: ejecutar y evidenciar el despliegue

App Streamlit + API (Flask/FastAPI) + Postman + OpenAPI

### Paso 0. Preparar la carpeta despliegue_semana06/

Reúne en una misma carpeta estos archivos (los tres primeros salen del Laboratorio 06; si usaste el notebook de despliegue aplicado, descárgalos desde el panel de archivos de Colab):

- modelo_produccion.keras, escalador_produccion.pkl y version_info_produccion.json
- api.py (Flask, de la guía) y api_fastapi.py (FastAPI, del notebook de despliegue aplicado)
- app_streamlit.py, Semana06_API_Prediccion_Lluvia.postman_collection.json y openapi.json

```
pip install tensorflow scikit-learn flask fastapi "uvicorn[standard]" streamlit
```

> ⚠️ **Atención**
>
> No levantes api_fastapi_esquema.py como servidor: solo sirve para generar openapi.json y sus endpoints son marcadores (devuelven error 500). El servidor FastAPI real es api_fastapi.py.

### Paso 1. App Streamlit (frontend)

```
streamlit run app_streamlit.py
```

Se abre en http://localhost:8501. Ingresa temperatura máxima, mínima y viento, y pulsa "Predecir lluvia de mañana". Si faltan los archivos del modelo, la app lo indica en pantalla.

### Paso 2. Levantar las APIs y probar con Postman

Abre dos terminales dentro de semana06/ y deja ambas corriendo:

```
python api.py                          # Flask, puerto 5000
uvicorn api_fastapi:app --port 8000    # FastAPI, puerto 8000
```

En Postman: Import → selecciona la colección .postman_collection.json → Run collection.
Resultados esperados:

| Solicitud | Código esperado | Qué demuestra |
|---|---|---|
| POST /predecir válida (Flask y FastAPI) | 200 | El modelo responde con predicción y probabilidad. |
| POST /predecir con campo faltante (Flask) | 400 | Validación manual con mensaje propio. |
| POST /predecir con campo faltante o tipo inválido (FastAPI) | 422 | Validación automática con pydantic. |
| GET /salud (Flask y FastAPI) | 200 | Monitoreo: la API está arriba y el modelo cargado. |

## Paso 3. Contrato de la API (openapi.json)

- En Postman: Import → openapi.json (crea la colección a partir del contrato).
- En editor.swagger.io: File → Import file, para ver la documentación interactiva.
- Con FastAPI corriendo: abre http://127.0.0.1:8000/docs (genera lo mismo automáticamente).

## Evidencias del despliegue

Pega aquí una captura por cada recuadro y escribe una línea que explique qué se observa.

### Evidencia 1 — App Streamlit con una predicción



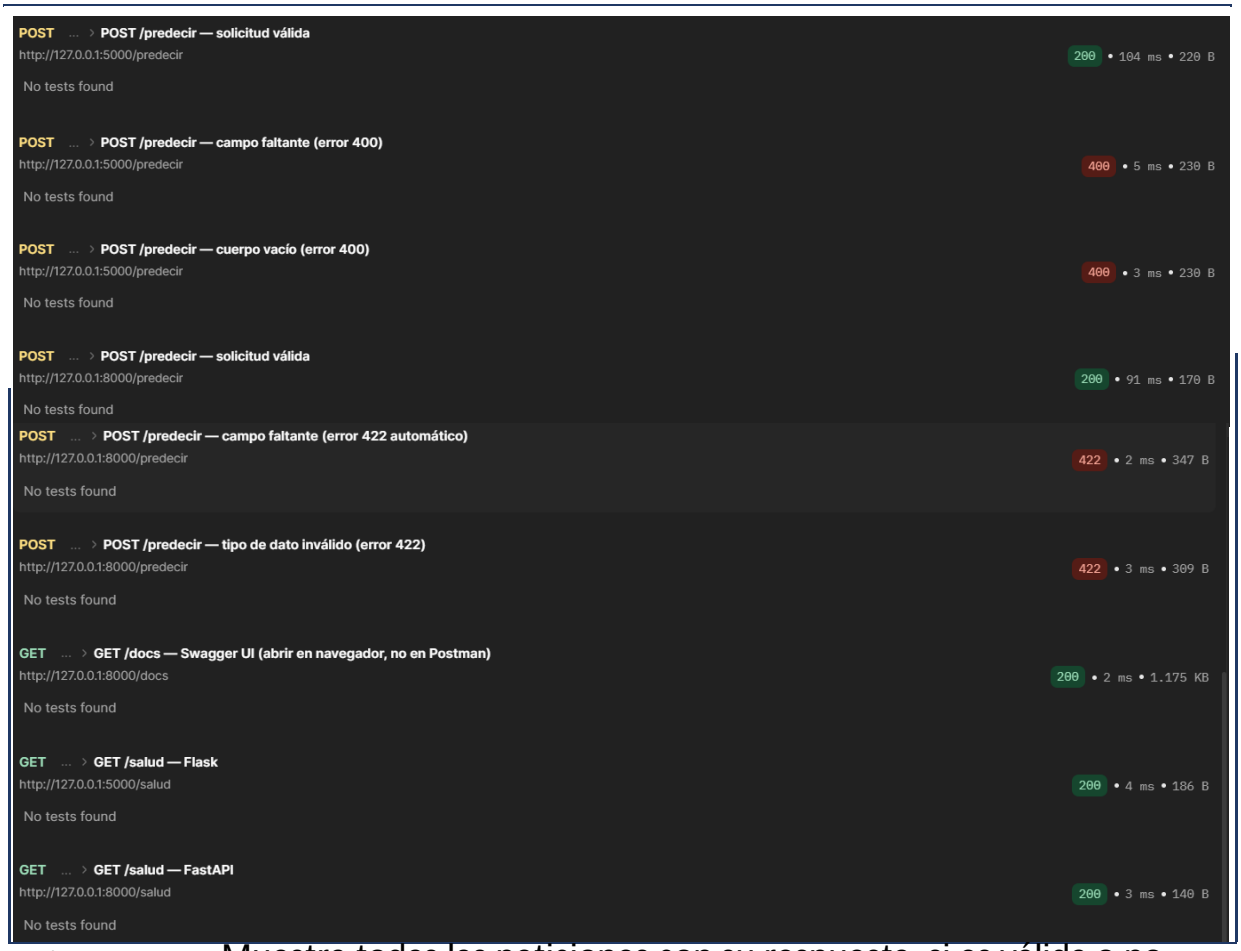

Qué se observa: Una página simple haciendo una predicción meteorológica.

### Evidencia 2 — Postman: respuesta 200 y respuesta de error (400 o 422)



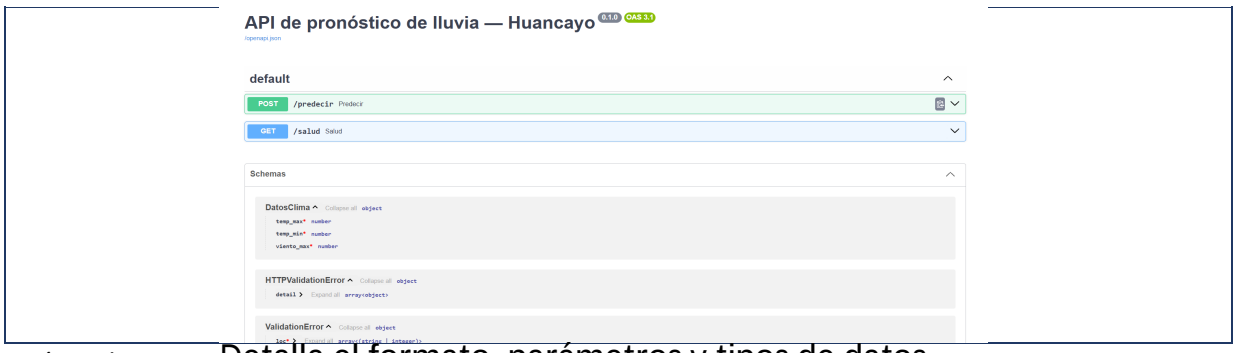

Qué se observa: Una solicitud que cumple con el formato json (200) y uno que no lo cumple (400)

## Evidencia 3 — Postman: resultado de "Run collection"




Qué se observa: Muestra todas las peticiones con su respuesta, si es válida o no.

## Evidencia 4 — Swagger UI (/docs) o editor.swagger.io con openapi.json




Qué se observa: Detalla el formato, parámetros y tipos de datos.

**Reflexión (2 a 3 líneas)**

¿Qué diferencia observaste entre el error 400 de Flask y el 422 de FastAPI, y cuál preferirías en un proyecto real? _____

El error 400 es una solicitud mal formada, en cambio el error 422 es una solicitud bien formada sintácticamente pero no semánticamente. Es preferible Fast API para no tener que hacer validaciones manuales.